计算机系统基础 PA4-1 实验报告

实验人：葛家韬（241220071）
授课教师：汪亮 苏丰

PA4-1 异常与中断

实验问题

1. 详细描述从测试用例中的`int $0x80`开始一直到`HIT_GOOD_TRAP`为止的详细的系统行为（完整描述控制的转移过程，即相关函数的调用和关键参数传递过程），可以通过文字或画图的方式来完成；

- 用户程序执行`int $0x80`。
- nemu通过opcode对该指令进行指令译码，调用对应的函数，进行eip + 2，同时将EFLAGS CS EIP压栈
- 查询IDTR，得到 0x80对应的门描述符，加载CS跳转到对应的代码段。
- 将错误码压栈，跳转到`asm_do_irq`，用`pushal`保存通用寄存器，`pushl %esp`并调用`do_syscall`，根据错误码进行分发，此处为输出操作，因此分发到`fs_write`，通过4-2中实现的模拟串口输出字符串。
- 系统调用后执行`iret`，弹出EFLAGS CS EIP，回到用户态执行。
- 用户把eax置为0，实现`HIT GOOD TRAP`。

2. 在描述过程中，回答`kernel/src/irq/do_irq.S`中的`push %esp`起什么作用，画出在`call irq_handle`之前，系统栈的内容和`esp`的位置，指出`TrapFrame`对应系统栈的哪一段内容。

```
+---------------------------+
| EFLAGS                    |
+---------------------------+
| CS                        |
+---------------------------+
| EIP                       |
+---------------------------+
| Error Code                |
+---------------------------+
| IRQ No                    |
+---------------------------+
| EAX                       |
+---------------------------+
| ECX                       |
+---------------------------+
| EDX                       |
+---------------------------+
| EBX                       |
+---------------------------+
| Old ESP                   |
+---------------------------+
| EBP                       |
+---------------------------+
```

```
| ESI                        |
+----------------------------+
| EDI                        |
+============================+
| %esp 的值 (即参数 tf)      |  <---- %esp
+----------------------------+
```

`pushl %esp` 把当前esp传给 `irq_handle`，使其获得一个指向 `TrapFrame` 的指针。
`TrapFrame` 是从edi开始到eflags的区域。

3.详细描述NEMU和Kernel响应时钟中断的过程和先前的系统调用过程不同之处在哪里？相同的地方又在哪里？可以通过文字或画图的方式来完成。

不同：系统调用由用户程序通过执行 int $0x80 指令同步触发，NEMU 在指令执行阶段即可直接从操作数中获取中断号；而时钟中断则由外部设备异步产生，NEMU 会在每条指令执行结束后调用 do_intr() 轮询中断引脚，当检测到中断条件满足时，再向 PIC 查询对应的中断号。
相同：两者最终都会调用 raise_intr 函数，执行统一的硬件上下文保存操作（依次压入 EFLAGS、CS 和 EIP），并通过 IDT 查表完成跳转。进入内核后，它们汇聚到同一个汇编入口，构建相同的 TrapFrame 结构，并统一调用 C 函数 irq_handle 进行中断分发处理，最后均通过 iret 指令恢复现场并返回执行。

课程知识点

- IDT/IDTR 的设置、trap gate 与 interrupt gate 的区别及 IF 处理
- 硬件压栈与软件保存现场形成 TrapFrame 的约定
- `int` 系统调用与外部中断的路径差异（同步/异步、IF 检查、PIC 参与）
- PIC 查询与 CPU 中断响应的协作流程

实验过程（由 AI 总结）

1. 在 `include/config.h` 打开 `IA32_INTR`、`HAS_DEVICE_TIMER` 等宏，`make clean` 后重新编译；`init_cpu` 初始化 `idtr`、`intr` 并调用 `i8259_init`。
2. 在 NEMU 补全 `lidt/cli/sti/int/pusha/popa/iret` 以及 `raise_intr`，并在 `exec()` 循环尾调用 `do_intr()` 轮询可屏蔽中断。
3. 在 Kernel 初始化时重建 GDT、填充 IDT（`init_idt`）并 `sti` 开中断，`asm_do_irq` / `irq_handle` 负责现场保护与分发。
4. 测试 `hello-inline` 验证自陷系统调用打印正确；触发定时器中断，定位并去掉理解流程用的 `panic`。

实验心得与疑问

- `TrapFrame` 布局完全依赖汇编栈约定，逐行对应推导后才敢使用，学会了用栈布局去验证结构体定义。
- 外部中断的插入点在每条指令之后，和同步的 `int` 路径共享 `asm_do_irq` / `irq_handle`，体现了硬件与内核解耦的设计。

参考资料及声明

- 课程提供的 `4-1guide.md`、课件与 i386 手册 Chapter 9
- PA 框架代码（`pa_nju` 目录）
- 报告表述与排版参考了 AI 工具建议，核心实验与结论均由本人完成。